

RPG SOLO

TRANSMISSOR DE CONTEXTO

DO PROJETO

Guia central para novas conversas, continuidade entre frentes de trabalho e atualização semanal do desenvolvimento.

VERSÃO	DATA-BASE	BRANCH
1.0	4 de agosto de 2026	main

1. Como usar este documento

Este PDF é o ponto de partida de qualquer nova conversa sobre o RPG Solo. Ele não substitui o código, o histórico do Git nem a checklist: ele explica como essas fontes se relacionam e oferece um retrato legível do estado do produto.

Instrução inicial para uma nova conversa

Antes de sugerir ou alterar código, leia este documento, confira a branch e o estado local do repositório e consulte a checklist mais recente. Verifique se a estrutura proposta já existe, mesmo parcialmente. Não presuma que uma função precisa ser criada antes dessa busca.

Ordem de confiança das fontes

Fonte	Como interpretar
1. Código atual	Fonte de verdade sobre o que realmente está implementado.
2. Git	Indica versão confirmada, mudanças locais ainda não consolidadas e histórico das decisões.
3. Checklist MVP	Fonte de verdade sobre escopo, critérios de conclusão e progresso formal.
4. Este PDF	Síntese editorial para orientação rápida; deve ser atualizado semanalmente.
5. Conversas	Contexto temporário. Decisões duradouras devem migrar para código, checklist ou este documento.

Protocolo de atualização semanal

- Cada conversa registra somente sua contribuição confirmada: arquivos afetados, comportamento entregue, testes realizados, decisões e pendências.
- No fim da semana, uma conversa consolidadora compara essas contribuições com o código e com a checklist antes de atualizar este documento.
- Nenhum item vira 'concluído' apenas porque o código existe; banco, interface, regra, persistência e teste aplicáveis devem estar comprovados.
- O PDF recebe nova versão e data-base. A versão anterior pode ser arquivada para rastreabilidade.

2. Visão do produto e recorte do MVP

O RPG Solo é uma aplicação web para criar personagens, escolher aventuras e jogar uma experiência narrativa solo com ficha integrada, decisões, testes, combate em grid e memória da aventura. A hipótese do MVP é permitir RPG solo com menos preparação e uma apresentação mais imersiva.

Recorte funcional

Pilar	Definição atual
Personagem	Criação guiada e salvamento local de personagem de nível 1.
Cena	Situação jogável com contexto, objetivos, escolhas e caminhos internos.
Consequência	Testes, escolhas e combate alteram a narrativa.
Memória	Estado da aventura, ficha e consequências precisam persistir.
Criação	Ferramentas pequenas para conteúdo e expansão futura, sem framework no MVP.

Escopo de personagem

- Classes priorizadas, nesta ordem: Guerreiro nível 1, Mago nível 1, Ladino nível 1 e Clérigo nível 1.
- Criação absolutamente completa para o recorte ofertado: atributos pelo método 4d6, antecedente, espécie, classe, equipamento, proficiências, habilidades, talentos e magias aplicáveis.
- Tendência foi removida do escopo. Outros métodos de geração de atributos também não fazem parte do MVP neste momento.
- Experiência inicial do MVP: batalha concede 100 XP na vitória, para validar a mecânica antes de balanceamento avançado.

Diretrizes de produto

- Textos criativos devem ser neutros quanto a gênero sempre que possível.
- O jogador conhece seus próprios dados; informações internas de inimigos não devem ser expostas sem motivo narrativo.
- Mensagens exibidas em combate devem vir dos bancos narrativos, evitando frases soltas em arquivos de regra e interface.
- A experiência deve funcionar sem uso do console no critério final do MVP.

3. Arquitetura atual

Tecnologia: HTML, CSS e JavaScript sem framework nem etapa de compilação. Personagens são persistidos em localStorage. A ficha compartilhada é carregada por servidor local e reutilizada nas três instâncias principais.

Área	Responsabilidade
Entrada e catálogo	index.*, aventuras-disponiveis.*, meus-personagens.*
Criação	criacao-personagem.* coordena as etapas e prévias do personagem.
Ficha compartilhada	ficha-personagem.*, renderizador-ficha.js e inicializador-pagina-ficha.js.
Visualização	ver-personagem.* mostra personagem salvo e exporta ficha.
Aventura	aventuras.* coordena cenas, escolhas, testes e integração com combate.
Combate	combate.js contém regras; interface-combate.js apresenta ações; confirmacao-combate.js trata confirmações.
Dados	dados.js e dados.css controlam a caixa, solicitações, validação e resultados.
Estado	estado-jogo.js e personagem-dados.js normalizam, guardam e migram dados.
Efeitos	banco-efeitos.js descreve efeitos; motor-efeitos.js interpreta operações genéricas.
Narrativa	mensagens-narrativas.js centraliza textos de interface; narracao-combate.js gera variações de acontecimentos.
Conteúdo	banco-*.js separa aventuras, NPCs, classes, antecedentes, espécies, equipamentos, habilidades, talentos, maestrias, magias, perícias e idiomas.

Regra de separação

Dados descrevem; motores interpretam; interfaces exibem

Os bancos não devem executar lógica específica de interface. Motores não devem conter textos finais espalhados. Interfaces não devem recriar regras já existentes. Antes de adicionar uma função exclusiva, procurar uma operação genérica ou uma implementação parcial.

Ficha como componente

A ficha foi centralizada para ser chamada durante a criação, em Ver personagem e na aventura/combate. Formatação e comportamento compartilhados devem permanecer no componente; páginas hospedeiras devem fornecer contexto e espaço, não duplicar a ficha.

4. Estado confirmado do desenvolvimento

Progresso formal

Checklist integrada de 3 de agosto de 2026: 297 de 743 itens (39,97%). Esse valor é a linha de base auditada e não inclui automaticamente as mudanças locais ainda não consolidadas.

Entregas já estabelecidas

- Home reformulada com acesso a criação, personagens salvos e aventuras disponíveis.
- Criação e persistência local de personagens; seleção de personagem antes da aventura.
- Avatares e frames combináveis, com prévia na ficha e uso como token de batalha.
- Ficha compartilhada nas três instâncias principais.
- Banco de aventuras com cenas, escolhas, etapas, testes sequenciais, retornos e consequências.
- Tela de combate em grid amplo, pan e zoom, tokens arrastáveis e posições iniciais configuráveis.
- Iniciativa, ordem de turnos, movimento, ação, ação bônus, reação, alcance, ataque, dano, acerto crítico, vitória e derrota em estágio funcional.
- Painel flutuante de ações, fila visual de iniciativa, ação atual e histórico limitado/expansível.
- Motor genérico de efeitos iniciado; Atacante Selvagem e Segundo Fôlego usados como pilotos.
- Antecedentes existentes concluídos formalmente na checklist, incluindo bônus de atributos.
- Ficha e personagem incluem estruturas voltadas à progressão futura e níveis por classe.

Pontos parcialmente concluídos

Frente	Situação
Economia de ações	Base funcional; ainda faltam recursos, concentração e critério de interface/estado sempre sincronizados.
Motor de efeitos	Formato e parte dos gatilhos existem; faltam alcance, área, duração, condições e ampla cobertura de gatilhos.
Armas	Modelo e ataques básicos funcionam; banco completo, testes de todas as armas e maestrias ainda faltam.
Talentos	Atacante Selvagem é piloto; faltam requisitos, gatilhos e testes de todos os talentos ofertados.
Magias	Estrutura inicial existe; conjuração completa, áreas, concentração, espaços e magias piloto ainda faltam.
IA inimiga	Seleciona alvo e ataque; perseguição, decisões táticas, recursos e condições precisam amadurecer.
Persistência pós-combate	Ainda não consolida integralmente PV, recursos, condições, munição e recompensas no personagem salvo.

5. Trabalho local ainda não consolidado

Na data-base deste documento, a branch é main e existem alterações locais em seis arquivos. Uma nova conversa deve preservar essas mudanças e não presumir que o último commit representa o estado executado no navegador.

Arquivo modificado	Contexto conhecido
aventuras.html	Mudanças locais relacionadas à interface/integração atual.
aventuras.js	Solicitação de iniciativa conectada à caixa de dados e ajustes recentes de fluxo.
combate.js	Participante do combate passa a precisar preservar nível e níveis por classe.
dados.js	Validação de combinação solicitada, rolagens livres, resultado com modificador e aviso narrativo em revisão.
interface-combate.js	Mudanças locais no fluxo de ações e efeitos.
motor-efeitos.js	Resolução de modificadores e operações de efeitos em desenvolvimento.

Decisão atual da caixa de dados

- Sem solicitação: o usuário pode lançar qualquer dado; a rolagem não altera o jogo.
- Com solicitação: qualquer combinação pode ser fisicamente rolada, mas somente a combinação exata resolve a solicitação.
- Combinação incorreta: anima normalmente, exibe mensagem vinda de mensagensNarrativas.dados.erroRolagem e mantém a solicitação pendente.
- Resultado aceito: mostra lançamento + modificador = total final; crítico usa a fórmula narrativa própria.
- A iniciativa deve solicitar exatamente 1d20 com o bônus do personagem.
- Segundo Fôlego deve rolar 1d10 + nível de Guerreiro; o participante do combate precisa conservar nível e níveisPorClasse.

Atenção imediata

Antes do próximo commit, testar iniciativa com 1d8, 2d20 e 1d20; testar uma rolagem livre; testar Segundo Fôlego de Guerreiro nível 1; confirmar que textos e contas vêm de mensagens-narrativas.js. Só então consolidar o estado e atualizar a checklist.

6. Próxima rota recomendada

A estratégia acordada é terminar uma classe de nível 1 por vez, com todas as dependências necessárias, testando cada camada antes de ampliar o banco. A ordem é Guerreiro, Mago, Ladino e Clérigo.

Ordem	Critério prático
1. Estabilizar dados	Concluir e testar solicitações, rolagens livres, modificadores e textos centralizados.
2. Guerreiro N1	Fechar criação, equipamentos, Segundo Fôlego, maestria e talentos aplicáveis; terminar combate completo.
3. Persistência	Salvar PV, recursos e alterações do combate; recarregar sem perder estado.
4. XP	Conceder 100 XP uma única vez na vitória, atualizar ficha e sinalizar progressão.
5. Mago N1	Implementar conjuração piloto: ataque, salvaguarda, área, cura quando aplicável, concentração, truque e espaço.
6. Ladino N1	Fechar recursos e mecânicas próprias, com integração ao motor genérico.
7. Clérigo N1	Fechar magia, cura, recursos e escolhas próprias.
8. A Fuga	Jogar do início ao fim com cada classe, sem console, e retornar corretamente à narrativa.

Pendências estruturais do MVP

- Completar motor genérico de efeitos e economia de ações.
- Finalizar armas necessárias, maestrias, talentos e habilidades ofertadas.
- Construir magia básica, áreas de efeito e condições.
- Completar quatro classes no nível 1.
- Revisar IA inimiga e movimento por caminho visível.
- Persistir consequências de combate.
- Implementar XP e preparação da progressão.
- Encerrar combate e continuar aventura de forma robusta.
- Executar rodada final de testes, acessibilidade, responsividade e revisão editorial.

7. Regras de colaboração e implementação

- Por padrão, ensinar alterações locais passo a passo. Só editar diretamente quando o usuário pedir explicitamente.
- Nunca alterar GitHub, fazer commit, push, merge ou abrir PR sem aviso e autorização explícita.
- Respeitar o estilo de indentação espaçado adotado no projeto e enviar exemplos no mesmo padrão.
- Antes de criar nova função, banco ou responsabilidade, procurar implementações existentes e parciais em todo o projeto.
- Centralizar comandos de um mesmo elemento no mesmo bloco sempre que possível e evitar CSS corretivo acumulado ao fim dos arquivos.
- Preservar alterações locais do usuário e trabalhar apenas no escopo pedido.
- Validar sintaxe, HTML e comportamento proporcionalmente ao risco; informar claramente o que foi e o que não foi testado.
- Textos visíveis devem vir do banco apropriado. Código de regra não deve inventar redação final.

Critério de conclusão

Definição de pronto

Uma funcionalidade está pronta quando sua estrutura de dados, regra, interface, persistência e testes aplicáveis funcionam juntas, sem erro no console e sem depender de comandos manuais. Implementação parcial deve permanecer marcada como parcial.

Cuidados legais e de conteúdo

- O MVP usa material do SRD 5.2.1 sob CC BY 4.0, com atribuição registrada em CREDITS.md.
- Recursos visuais ainda precisam de origem, autoria e licença documentadas antes de distribuição ampla.
- Livros de referência servem para estudo e desenho interno; conteúdo não autorizado não deve ser presumido como publicável.

8. Registro semanal de contribuições

Copie este modelo para cada frente de trabalho. A atualização deve ser factual, curta e verificável.

Campo	Preenchimento
Semana	AAAA-MM-DD a AAAA-MM-DD
Conversa/frente	Nome curto da frente trabalhada
Objetivo	Resultado pretendido
Arquivos afetados	Lista dos arquivos realmente modificados
Entregue	Comportamentos concluídos e demonstrados
Testes	Cenários executados e resultados
Decisões	Escolhas arquiteturais ou de produto que devem persistir
Pendências	O que ficou incompleto, quebrado ou sem teste
Checklist	IDs/itens alterados e justificativa
Git	Branch e commit; registrar 'não consolidado' quando aplicável
Próximo passo	Uma ação concreta e pequena

Exemplo: contribuição atual

Campo	Registro
Frente	Validação e narrativa da caixa de dados.
Entregue	Iniciativa conectada a 1d20; rolagem incorreta não resolve solicitação; rolagem livre permitida.
Em ajuste	Usar mensagensNarrativas.dados para erro e fórmulas; garantir +1 do Segundo Fôlego.
Teste final	1d8 e 2d20 devem rolar e ser rejeitados; 1d20 aceita bônus; 1d10 + 1 cura.
Git	Mudanças locais na main, ainda sem consolidação registrada neste documento.

9. Checklist de abertura de uma nova conversa

- Ler este PDF por completo.
- Confirmar pasta do projeto, branch e alterações locais.
- Ler README.md e CREDITS.md quando a tarefa tocar escopo público ou licenças.
- Consultar a versão mais recente da checklist MVP e sua data-base.
- Localizar os arquivos da frente e buscar funções/estruturas existentes antes de propor código.
- Confirmar com o usuário se a conversa ensinará mudanças ou editará arquivos diretamente.
- Definir um próximo passo pequeno, testável e coerente com a rota de classes.
- Ao terminar, produzir o registro semanal de contribuição.

Resumo executivo para retomada

Onde estamos

O projeto já possui criação, personagens salvos, ficha compartilhada, aventuras ramificadas e um combate funcional em estágio intermediário. A prioridade imediata é estabilizar a caixa de dados e concluir o Guerreiro nível 1 de ponta a ponta. Depois vêm persistência e XP, seguidos de Mago, Ladino e Clérigo.

Fontes consultadas nesta versão

- Repositório local RPG Solo, branch main, estado observado em 4 de agosto de 2026.
- README.md e CREDITS.md do projeto.
- RPG Solo MVP Checklist - progresso integrado.xlsx, atualização de 3 de agosto de 2026.
- Histórico consolidado de decisões da conversa de desenvolvimento.

Fim da versão 1.0 - atualizar ao fim de cada semana de desenvolvimento.

RPG SOLO

SUPLEMENTO TÉCNICO E HISTÓRICO

Atualização cumulativa do Transmissor de Contexto
Do início da frente narrativa até a arquitetura proposta para câmera e mundo de combate

VERSÃO	DATA-BASE	STATUS DA FONTE
2.0	16 de agosto de 2026	PDF original preservado + histórico da conversa

IMPLEMENTADO	Comportamento descrito na conversa como incorporado e posteriormente conferido em commit/push.
CORRIGIDO	Defeito identificado, causa explicada e correção incorporada ou confirmada.
PROPOSTA	Decisão em discussão, desenho recomendado ou alteração explicitamente ainda não aplicada.

Nota de preservação

As dez páginas da versão 1.0 permanecem no início deste arquivo sem reescrita. Este suplemento acrescenta o histórico posterior e também detalha mudanças anteriores que a versão 1.0 apenas resumia. Onde o espelho local não contém os commits novos, a classificação se baseia nas revisões de commit registradas na conversa; o código atual do repositório continua sendo a fonte final de verdade.

10. Linha arquitetônica e limpeza estrutural

A regra que orientou a reorganização foi formalizada como **Dados descrevem; motores interpretam; interfaces exibem**. O objetivo não era somente dividir arquivos, mas impedir que conteúdo narrativo executasse DOM, que motores acumulassem frases finais e que páginas recriassem regras já existentes.

Separação de responsabilidades

- **banco-aventuras.js / aventuras.js**: conteúdo declarativo das aventuras - cenas, etapas, escolhas, testes, memórias, combates e destinos.
- **motor-aventura.js**: interpretação genérica das estruturas declaradas, incluindo testes, consequências e ataques automáticos de NPC.
- **estado-jogo.js**: criação, normalização e persistência do estado de execução; mantém progresso e memórias sem contaminar o banco.
- **narrador-aventura.js**: seleção e composição de texto narrativo, inclusive variações condicionadas por memórias.
- **aventuras.js**: orquestração da página, renderização, entrada em cenas/etapas, encaminhamento das escolhas e ponte com ficha, dados e combate.
- **combate.js**: regras e estado do combate; **interface-combate.js**: DOM, câmera e apresentação; **combate.css**: aparência da batalha.

Limpeza e remoções

A frente de limpeza removeu caminhos e arquivos legados que duplicavam responsabilidades, eliminou referências mortas, consolidou validações e reduziu a dependência entre a ficha salva e o CSS do criador. O princípio usado foi preservar uma implementação canônica por responsabilidade. A remoção não foi estética: evitou duas fontes de verdade para ficha, aventura, dados e câmera.

IMPLEMENTADO

A limpeza estrutural foi consolidada na linha de commits iniciada por "Dispara rotina de limpeza do projeto" e "Organiza estrutura e validações do projeto", integrada à main pelo PR de limpeza estrutural.

Efeito prático

```
banco / dados declarativos
  ↓
motor genérico
  ↓
interface / página hospedeira
  ↓
estado persistido
```

11. Modelo narrativo: cenas, etapas, escolhas, testes e memórias

A aventura deixou de depender de uma sequência rígida de blocos e passou a ser descrita como um grafo pequeno. Uma **cena** fornece contexto e pode iniciar combate, etapa ou escolhas. Uma **etapa** representa uma unidade interna: texto, teste, ataque narrativo, escolhas ou destino. Uma **escolha** conduz a outra etapa ou cena. **Memórias** registram consequências sem acoplar o conteúdo ao estado da interface.

```
cenaExemplo: {
  contexto: ['...'],
  etapaInicial: "primeiroDisparo",
  etapas: {
    primeiroDisparo: { ataqueNpc: { ... } },
    decidirRota: {
      escolhas: [
        { texto: "Subir", proximaEtapa: "escalar" },
        { texto: "Ir pelo chão", proximaCena: "torreChao",
          memorias: { origemTorreChao: "flechada" } }
      ]
    }
  }
}
```

Interpretação do fluxo

- ``exibirCena()`` mostra o contexto e verifica, em ordem controlada, combate, ``etapaInicial`` e escolhas.
- ``iniciarEtapa()`` resolve a etapa corrente e encaminha ataque de NPC, teste, escolhas ou destino.
- Testes usam esquemas explícitos: ``tipo: atributo``, ``salvaguarda`` ou ``pericia``, com o identificador correspondente e uma dificuldade.
- Resultados têm ``sucesso`/`fracasso`` ou ``acerto`/`erro``, cada qual podendo trazer texto, próxima etapa, próxima cena e memórias.
- Memórias permitem ao narrador escolher variações futuras sem duplicar cenas inteiras.

Mudanças por arquivo

IMPLEMENTADO

aventuras.js passou a reconhecer ``etapaInicial`` e a encaminhar ``ataqueNpc``; motor-aventura.js recebeu resolução genérica do ataque; estado-jogo.js sustenta progresso/memórias; narrador-aventura.js seleciona texto segundo o estado.

12. Ataques narrativos, crítico e queda

Ataque narrativo é um ataque de NPC executado dentro da aventura, antes ou fora do tabuleiro. O banco declara `npcId`, `ataqueId` e resultados; o motor busca os dados canônicos do NPC, rola ataque contra a defesa do personagem, aplica dano e encaminha o fluxo.

```
ataqueNPC: {  
  npcId: "guardaConde",  
  ataqueId: "arcoCurto",  
  dano: { substituirModificador: -3, minimo: 1 },  
  resultados: {  
    acerto: { proximaEtapa: "salvaguardaImpacto" },  
    erro: { proximaEtapa: "segundoDisparo" }  
  }  
}
```

Dano especial sem alterar o NPC

O segundo disparo precisava ser deliberadamente fraco. Em vez de editar `arcoCurto` globalmente, a etapa substitui apenas o modificador e impõe piso. Assim, o ataque normal continua `1d6 + 1`, enquanto o evento narrativo pode ser `1d6 - 3`, mínimo 1.

Crítico coerente com o combate

```
const subtotal = Number(resultadoDano.subtotal) || 0;  
const modificador = Number(resultadoDano.modificador) || 0;  
  
dano = acertoCritico  
  ? subtotal * 2 + modificador  
  : subtotal + modificador;  
  
dano = Math.max(configuracao.dano?.minimo ?? 0, dano);
```

CORRIGIDO

A primeira versão dobrava o total inteiro (`dados + modificador`). A revisão alinhou o ataque narrativo à regra do combate: dobra o subtotal dos dados e adiciona o modificador uma vez.

Queda e cadeia de testes

A Cena 04 encadeou disparo, dano, salvaguarda de Constituição, teste de Força para não cair/subir e Atletismo para o salto. Falhas podem levar à batalha; sucessos preservam a rota. Essa estrutura validou testes sequenciais, etapas automáticas e decisões intercaladas.

13. Cena 04: fluxo final e correções de integração

```
entrada / variação narrativa
↓
primeiroDisparo
  ■■ erro → segundoDisparo
  ■■ acerto → dano → salvaguarda CON
    ■■ sucesso → segundoDisparo
    ■■ fracasso → teste de Força
      ■■ sucesso → segundoDisparo
      ■■ fracasso → batalhairuas

segundoDisparo (1d6 - 3, mínimo 1)
↓
escolha: descer ao chão ou saltar
  ■■ chão → torreChao + memória
  ■■ salto → Atletismo CD 16
```

Falhas encontradas e resolvidas

- `etapaInicial` apontava para `primeiroAtaque`, mas a etapa se chamava `primeiroDisparo`.
- `exibirCena()` ainda não iniciava automaticamente a etapa inicial.
- `iniciarEtapa()` não interpretava `ataqueNpc`.
- O segundo disparo apontava para uma etapa inexistente (`pularAteTorreFlecha`) em vez da etapa de escolha real.
- A etapa de escolha também continha teste; como o motor priorizava teste, a decisão do jogador seria pulada. O teste foi mantido em etapa própria.
- O motor ignorava `configuracao.dano.substituirModificador` e aplicava o dano normal do arco.
- Marcadores editoriais `VER SE PRECISA` e inconsistência textual entre besta/arco foram sinalizados para revisão.

IMPLEMENTADO

Após os ajustes, a estrutura foi revisada como coerente e pronta para teste integral no navegador.

Estruturas de teste suportadas

```
// atributo
{ tipo: "atributo", atributoId: "forca", dificuldade: 15 }

// salvaguarda
{ tipo: "salvaguarda", atributoId: "constituicao", dificuldade: 14 }

// perícia
{ tipo: "pericia", periciaId: "atletismo", dificuldade: 16 }
```

14. Criação e evolução da tela de combate

A tela de combate evoluiu de um grid grande e quase autônomo para uma batalha vinculada a uma cena. A primeira cena mínima, `batalha1ruas`, foi criada para testar mapa, grid, jogador, dois guardas, câmera e transição narrativa.

Configuração por batalha

```
combate: {  
  mapa: "Imagens/Mapas/A Fuga/batalha1ruasn.webp",  
  jogador: { posicao: { coluna: 25, linha: 22 } },  
  inimigos: [{  
    npcId: "guardaConde", quantidade: 2,  
    posicoes: [  
      { coluna: 21, linha: 8 },  
      { coluna: 26, linha: 9 }  
    ]  
  }],  
  resultados: { vitoria: { ... }, derrota: { ... } }  
}
```

Mapa dinâmico

Cada batalha declara somente seu arquivo de mapa. `verificarCombateDaCena()` transporta `cena.combate.mapa` para a configuração e `iniciarCombateDaAventura()` aplica a imagem. Esse fluxo preserva a regra arquitetônica: o banco escolhe o recurso; a interface o apresenta.

```
banco-aventuras.js  
  cena.combate.mapa  
    ↓  
aventuras.js / configuração de combate  
    ↓  
aplicarMapaCombate(configuracao.mapa)  
    ↓  
mapa exibido sob grid e tokens
```

Separação de CSS

IMPLEMENTADO

Regras de batalha foram retiradas de aventuras.css e reunidas em combate.css, importado por aventuras.css. Aventura mantém narrativa, pergaminho e gavetas; combate.css mantém viewport, campo, grid, células, tokens, PV, comandos, iniciativa, histórico e responsividade.

15. Evolução dimensional: 40x30 → 48x36 → 48x27

O tabuleiro começou em **40 x 30**, com células de 64 px. Para padronizar mapas maiores e permitir visão aérea, foi proposta e implementada a passagem a **48 x 36**: mundo lógico de 3072 x 2304 e arte 2x de 6144 x 4608, ainda em 4:3.

$40 \times 30 \times 64 \text{ px} = 2560 \times 1920$
 $48 \times 36 \times 64 \text{ px} = 3072 \times 2304$
arte 2x: 6144 x 4608

Problema revelado pela tela real

Em viewport widescreen, um mapa 4:3 só pode cumprir uma de duas metas: aparecer inteiro com faixas laterais ou preencher 16:9 cortando topo/rodapé. A captura de teste mostrou que a fundação precisava mudar antes de produzir muitos mapas.

Novo padrão 16:9

arte: 6144 x 3456 (16:9)
grade: 48 x 27
 $6144 / 48 = 128 \text{ px por coluna da arte}$
 $3456 / 27 = 128 \text{ px por linha da arte}$

IMPLEMENTADO

A grade lógica e o CSS foram ajustados de 48x36 para 48x27 e a nova imagem 16:9 foi usada no teste. O mapa, a grade e os tokens passaram a camadas separadas dentro do elemento transformado pela câmera.

Motivo da mudança

48x27 mantém células quadradas, usa toda a resolução 16:9 e aproxima a proporção do mundo da proporção predominante das telas. A grade continua sendo lógica; o número 128 pertence apenas à correspondência da arte original e não deveria reger as regras do jogo.

16. HUD, ficha e caixa de dados

O combate passou a ser tratado como um mundo observado pela câmera, enquanto os controles formam um HUD independente. A câmera nunca deve transformar ações, dados, iniciativa, histórico ou ficha.

```
HUD (fixo na viewport)
■■■ ações
■■■ caixa de dados
■■■ iniciativa
■■■ histórico
■■■ ficha

MUNDO (transformado)
■■■ mapa
■■■ grid
■■■ tokens
```

Caixa de dados compartilhada

A caixa desaparecia porque estava dentro de `#visualizacaoAventura`, que recebe `hidden` ao entrar em combate. A solução reutiliza a mesma instância: um comentário marca o local original; no combate a caixa é anexada ao `document.body` e posicionada como janela flutuante; ao sair, volta ao marcador. Isso evita IDs duplicados e conserva o estado da rolagem.

CORRIGIDO

Caixa de dados movida dinamicamente para o HUD do combate e devolvida à aventura ao terminar.

Ficha do personagem

A gaveta estava fora da área escondida, mas o carregamento dependia de uma corrida de evento. A tentativa de usar `window.fichaPersonagemPronta.then(...)` ainda podia falhar com scripts `defer`, pois a promessa era criada apenas no `DOMContentLoaded`. A correção robusta chama diretamente a API do componente.

```
window.FichaPersonagem
  .iniciarComponente()
  .then(renderizarFichaDaAventura)
  .catch(function (erro) {
    console.error("Não foi possível carregar a ficha.", erro);
  });
```

CORRIGIDO

A ficha passou a esperar explicitamente a inicialização do componente, sem depender de capturar um evento no instante certo.

17. Registro dos problemas e correções

Mapa não carregava	Extensão <code>`.webp`</code> e/ou caminho divergente do arquivo <code>`.webp`</code> .	Caminho corrigido e conferido.
Grid visual e motor diferentes	CSS em 48x36, motor ainda em 40x30.	Dimensões do estado alinhadas; depois migradas para 48x27.
Imagem sumiu após separar o elemento de mapa	Tabuleiro posterior tinha fundo marrom opaco e cobria a imagem; camada antiga de background permanecia.	Fundo transparente, camada antiga removida, z-index explícito: mapa 0, grid 1, tokens 2.
Caixa de dados desapareceu	Era filha da visualização narrativa escondida.	Mesma caixa migra para body durante combate e retorna depois.
Ficha não carregava	Corrida entre evento/promessa e scripts defer.	Chamada direta a <code>FichaPersonagem.iniciarComponente()</code> .
Câmera duplicada	<code>`.obterZoomMinimoVisivel()`</code> foi recriada em <code>aventuras.js</code> apesar de existir em <code>interface-combate.js</code> .	Centralização da câmera na interface; remoção da duplicata.
Resize quebrava	Chamava <code>`.obterZoomMinimo()`</code> , função inexistente.	Usa <code>`.cameraCombate.zoomMinimo`</code> recém-calculado e reaplica limites.
Mapa cortado no mínimo	Uso de <code>`.Math.max`</code> para o encaixe.	Uso de <code>`.Math.min(largura, altura, zoomMaximo)`</code> para mostrar tudo.
Imagem borrada no zoom	Arte 6144x3456 era reduzida para mundo 3072x1728 e ampliada novamente via transform.	Ainda em andamento: mundo nativo e teto de zoom 1.

18. Estado implementado antes da nova proposta

O último estado confirmado na conversa, antes de interromper para repensar a qualidade, tinha os seguintes elementos:

- Combate acionado por uma cena real (`batalha1ruas`) com mapa declarado no banco.
- Arquivo de estilos `combate.css` separado.
- Grade 48x27 e mapa 16:9.
- Elemento de mapa separado do tabuleiro; grid transparente e tokens acima.
- Câmera aplicada ao contêiner correto, sem a função duplicada.
- Zoom mínimo recalculado no resize para permitir visão geral.
- Caixa de dados compartilhada disponível no combate.
- Ficha inicializada pela API do componente.
- HUD de ações, dados, iniciativa e histórico fora da transformação do mundo.

O que ainda não ficou perfeito

- O combate inicia longe demais, praticamente no zoom mínimo; o desejado é um enquadramento tático mais próximo.
- Ao aproximar, o mapa perde nitidez porque o mundo lógico intermediário já reduziu a imagem.
- O significado de célula ainda ficou acoplado a pixels em partes do CSS.
- A câmera ainda precisa separar formalmente viewport, mundo nativo e escala tática inicial.

PROPOSTA

Não considerar concluídas as mudanças de 6144x3456 no mundo, grid `1fr`, `zoomMaximo = 1` e zoom inicial por colunas visíveis. A conversa foi interrompida antes de aplicá-las.

19. Proposta atual: viewport, mundo nativo e grade proporcional

A arquitetura proposta abandona a tentativa de resolver nitidez aumentando a célula de 64 para 128. Resolução da arte, grade lógica e zoom passam a ser três dimensões independentes.

```
CAMPO DE COMBATE / VIEWPORT
100vw × 100vh, overflow hidden
  ■ observa
  ▼
MUNDO DE COMBATE
6144 × 3456
■■■■ imagem do mapa 6144 × 3456
■■■■ grade lógica 48 × 27 proporcional
■■■■ tokens nas mesmas coordenadas lógicas

HUD: fora do mundo e fora do transform
```

Estrutura HTML conceitual

```
<div class="campo-combate">
  <div id="cameraCombate" class="mundo-combate">
    <img id="imagemMapaCombate"
      class="imagem-mapa-combate" alt="">
    <div id="tabuleiroCombate"
      class="tabuleiro-combate"></div>
  </div>
</div>
```

Observação de nomenclatura

O elemento hoje chamado `cameraCombate` é, na prática, o **mundo transformado**. A câmera propriamente dita é a viewport. Renomear classes pode tornar a relação mais clara, mesmo que o ID seja preservado por compatibilidade.

PROPOSTA	Mudar o mundo para a resolução nativa da arte e manter mapa, grade e tokens como camadas sincronizadas dentro dele.
----------	---

20. Grade lógica com `1fr`

A grade não deve dizer quantos pixels possui uma célula. Ela apenas divide o mundo em 48 colunas e 27 linhas. Se o mapa futuro tiver outra resolução, a mesma lógica permanece válida.

```
.tabuleiro-combate {
  position: absolute;
  inset: 0;
  display: grid;
  grid-template-columns: repeat(48, 1fr);
  grid-template-rows: repeat(27, 1fr);
  background-color: transparent;
}

.imagem-mapa-combate {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: fill;
}
```

Consequência conceitual

6144 × 3456	= resolução da arte
48 × 27	= espaço lógico do jogo
1fr	= divisão proporcional do mundo
zoom	= somente a escala da observação

Tokens e regras

Tokens continuam referenciando coluna e linha; alcance, movimento e posições não mudam. O motor não precisa saber se uma célula ocupa 64, 128 ou 173 pixels. Apenas a interface converte a célula lógica para a fração correspondente do mundo.

Mapa configurável

```
mapa: {
  imagem: "Imagens/Mapas/A Fuga/batalha1ruas.webp",
  largura: 6144,
  altura: 3456,
  colunas: 48,
  linhas: 27
}
```

A configuração de dimensões pode ser omitida enquanto todos os mapas obedecerem ao padrão. Mantê-la no futuro permitiria mapas de outras resoluções sem alterar o motor.

PROPOSTA

Primeiro passo recomendado: tornar o mundo 6144x3456 e substituir células fixas por frações, sem trocar a imagem atual nem alterar regras.

21. Modelo de zoom: mínimo, inicial e máximo

O modelo separa três valores com funções diferentes.

Zoom mínimo - visão geral

```
zoomMinimo = Math.min(  
    larguraViewport / larguraMundo,  
    alturaViewport / alturaMundo  
);
```

Esse valor garante que o mapa inteiro caiba. Em uma área com cerca de 1200 px de largura observando um mundo de 6144 px, o mínimo fica próximo de 0,20.

Zoom inicial - enquadramento tático

O combate não deve começar necessariamente no mínimo. Em vez de multiplicar o mínimo por um fator arbitrário, recomenda-se definir quantas colunas devem estar visíveis. Por exemplo, mostrar aproximadamente 24 das 48 colunas produz uma escala tática consistente em telas diferentes.

```
const cameraCombate = {  
    deslocamentoX: 0,  
    deslocamentoY: 0,  
    zoom: 1,  
    zoomMinimo: 0,  
    zoomMaximo: 1,  
    colunasVisiveisInicialmente: 24  
};
```

Zoom máximo - resolução nativa

Com o mundo em 6144x3456, `zoom = 1` significa um pixel CSS para cada pixel lógico/nativo. O máximo não deve passar de 1: ampliar além disso não revela detalhe novo e volta a causar interpolação.

```
faixa típica em uma tela comum  
0,20  mapa inteiro  
0,40  visão inicial tática  
0,60  aproximação  
0,80  detalhe  
1,00  resolução nativa
```

PROPOSTA

Separar `zoomMinimo`, `zoomInicial` e `zoomMaximo`; calcular o inicial por escala tática, não por `zoomMinimo \* 2`.

22. Por que ocorre blur e o limite real de qualidade

A arte possui 6144x3456, mas a implementação testada a encaixava em uma base lógica de 3072x1728. O navegador reduzia a imagem e depois o `transform: scale(...)` ampliava essa representação. Essa reamostragem intermediária explica o aspecto borrado.

```
IMPLEMENTAÇÃO TESTADA
6144 × 3456 (arquivo)
  ↓ redução
3072 × 1728 (mundo lógico)
  ↓ scale()
imagem ampliada e interpolada

PROPOSTA
6144 × 3456 (arquivo e mundo)
  ↓ redução apenas para visão distante
zoom cresce até 1
  ↓
resolução original progressivamente utilizada
```

O que não usar

``image-rendering: pixelated`` e ``crisp-edges`` são inadequados para um mapa pintado: preservariam blocos ou arestas, mas deixariam telhados, pedras e texturas serrilhados.

Limite físico

Nenhum sistema mantém detalhe infinito em qualquer zoom. A meta correta é não ampliar acima da resolução nativa. Dentro de ``zoomMínimo ... 1``, a imagem 6144x3456 tem resolução suficiente para telas atuais; acima de 1 só existe interpolação.

PROPOSTA

Eliminar a base 3072x1728 e limitar a aproximação ao tamanho nativo. Esta é a correção estrutural para o blur; não foi implementada na conversa.

23. Próximos passos recomendados e critérios de teste

A implementação deve ser incremental para separar erros de layout, matemática e interação.

Passo 1 - mundo nativo e grade proporcional

- Definir o mundo em 6144x3456.
- Manter imagem, grid e tokens dentro do mesmo elemento transformado.
- Trocar ``64px`/`128px`` por ``repeat(48, 1fr)`` e ``repeat(27, 1fr)``.
- Confirmar as camadas: mapa 0, grid 1, tokens 2; HUD fora.

Passo 2 - matemática da câmera

- Calcular mínimo com ``Math.min`` a partir da viewport e do mundo.
- Fixar máximo em 1.
- Recalcular limites no resize sem saltar abaixo do mínimo.
- Limitar pan para impedir que o mundo desapareça além das bordas.

Passo 3 - enquadramento inicial

- Calcular o zoom inicial a partir de cerca de 24 colunas visíveis.
- Centralizar no jogador, no centro do grupo ou em um foco configurável da batalha.
- Permitir afastar até a visão geral e aproximar até a resolução nativa.

Testes de aceitação

- Ao abrir, a escala deve se aproximar da segunda captura do usuário, não mostrar o mapa inteiro por padrão.
- Ao afastar, os 48x27 espaços devem caber completamente.
- Ao aproximar, mapa, linhas e tokens devem permanecer perfeitamente alinhados.
- Nenhum painel do HUD deve mover ou escalar.
- Resize em 1366x768, 1920x1080 e 2560x1440 deve preservar limites e escala tática.
- A imagem não deve ser ampliada acima de 1; a nitidez deve melhorar progressivamente ao aproximar.

24. Commits recentes e estado de continuidade

A conversa registrou sucessivos ciclos de “alterar localmente → commit/push → revisão”. Os nomes/ordem completos dos commits posteriores não estão presentes no espelho local; portanto, o registro abaixo usa apenas identificadores textuais confirmados na conversa e não inventa hashes.

Registros confirmados

- Commit referido como “**Mudanças mapa de combate**”: introduziu a base 48x36 no CSS e o mapa dinâmico em evolução.
- Commits posteriores corrigiram caminho ``.webp``, alinharam motor e CSS, removeram câmera duplicada, corrigiram resize e migraram a caixa de dados.
- Um commit de separação de camadas adicionou ``cameraCombate`` no contêiner correto, um elemento próprio de mapa e removeu a aplicação antiga do mapa como background.
- Correção subsequente tornou o tabuleiro transparente e explicitou z-index, restaurando a imagem encoberta.
- A mudança para mapa 16:9 e grade 48x27 foi aplicada e testada antes da discussão de nitidez.

Estado da fonte local

O clone ``RPG-Solo-inspecao`` disponível neste ambiente termina em commits anteriores da ficha e limpeza estrutural, não contendo os arquivos narrativos/combate mais recentes. Por isso, este suplemento preserva a distinção entre **implementado segundo revisão da conversa** e **proposta ainda não aplicada**. Antes de retomar código, sincronizar o repositório real e conferir ``git status``, ``git log``, ``banco-aventuras.js``, ``aventuras.js``, ``motor-aventura.js``, ``estado-jogo.js``, ``narrador-aventura.js``, ``interface-combate.js`` e ``combate.css``.

Ponto exato de retomada

PROPOSTA

Não aumentar células para 128px como solução isolada. Retomar pela arquitetura viewport → mundo nativo 6144x3456 → grid 48x27 em 1fr → zoom mínimo/inicial/máximo.

Decisão duradoura

48 × 27 é lógica de jogo.
6144 × 3456 é resolução da arte.
zoom é apenas câmera.
HUD não pertence ao mundo transformado.

25. Checklist para a próxima conversa

- Ler primeiro as 10 páginas originais e este suplemento.
- Sincronizar e inspecionar o repositório real; o espelho local pode estar incompleto.
- Não reimplementar estruturas já criadas para cenas, etapas, escolhas, testes, memórias ou ataque de NPC.
- Preservar o cálculo de crítico: dados dobrados, modificador aplicado uma vez.
- Preservar a caixa de dados única e a inicialização explícita da ficha.
- Manter toda a câmera em interface-combate.js; aventuras.js apenas orquestra entrada/saída da batalha.
- Confirmar o estado atual de 48x27 e das camadas antes de editar.
- Tratar mundo nativo, grid 1fr e zoom máximo 1 como proposta até ver o código sincronizado.
- Implementar e testar em três passos; não misturar redimensionamento do mundo, cálculo do zoom e enquadramento inicial num único salto.
- Depois da câmera, revisar posições do jogador/guardas e retorno de vitória/derrota à narrativa.

Resumo executivo

O projeto já possui uma base narrativa modular, memória de aventura, testes sequenciais, ataques automáticos de NPC, combate acionado por cena, mapa dinâmico, HUD compartilhado e grid 16:9. O bloqueio atual é visual e arquitetônico: a imagem de alta resolução está sendo reduzida antes do zoom. A próxima intervenção deve separar definitivamente resolução da arte, grade lógica e câmera, sem alterar as regras de combate.

FIM DO SUPLEMENTO

Versão preparada para continuidade técnica. O documento original permanece anexado no início.

RPG SOLO

SUPLEMENTO DE STATUS E ROTA DE LANÇAMENTO

Atualização cumulativa do Transmissor de Contexto após a reestruturação modular e a auditoria da checklist do MVP.

VERSAO	DATA-BASE	BRANCH	CHECKLIST
3.0	18 de agosto de 2026	main	146 de 394 (37,06%)

Regra de leitura desta atualização

O percentual formal mede itens concluídos da checklist. Ele não equivale diretamente à maturidade técnica nem à distância de uma versão vertical jogável. O código atual continua sendo a fonte de verdade.

Fontes desta versão

- Repositório local na branch main, sincronizado com origin/main em 18 de agosto de 2026.
- Checklist atualizada: RPG Solo MVP Checklist - atualizado 2026-08-18.xlsx.
- Testes manuais confirmados durante a reestruturação e o desenvolvimento do combate.
- Versões 1.0 e 2.0 deste Transmissor, preservadas nas páginas anteriores.

26. Panorama atual do desenvolvimento

O projeto deixou de ser apenas um protótipo técnico e passou a constituir um produto jogável incompleto. A base de criação, aventura, dados e combate existe; o trabalho principal agora é fechar o ciclo de jogo, persistir consequências e completar o conteúdo ofertado.

Progresso e maturidade

Medida	Estimativa	Interpretação
Checklist formal	37,06%	146 de 394 itens concluídos com evidência.
Fundação técnica	65% a 70%	Estimativa editorial: arquitetura e sistemas-base já estão estabelecidos.
Vertical jogável com Guerreiro	70% a 80%	Estimativa editorial: falta principalmente fechar persistência, recursos e aventura.
MVP formal com quatro classes	40% a 50%	Estimativa editorial: Mago, Ladino e Clérigo ainda ampliam muito o escopo.

Entregas confirmadas desde o suplemento anterior

- Câmera, mapa, grid e tokens foram estabilizados no padrão de batalha adotado; pan, zoom e limites funcionam no teste atual.
- A caixa de dados voltou a mostrar modificadores e resultados e permite excluir lançamentos com o botão direito sem abrir o menu do navegador.
- Iniciativa, ataques, dano, crítico, turnos, vitória, derrota e Segundo Fôlego foram testados manualmente.
- Persistência de personagens foi centralizada em personagem-dados.js.
- Dados, câmera, renderização, HUD, comandos, narrativa, fluxo de combate, regras de criação e magias foram separados em módulos próprios.
- A correção do ataque narrativo voltou a comparar corretamente o resultado com a CA do alvo.
- A sintaxe dos arquivos JavaScript foi verificada após a reestruturação.

Estado do repositório

A branch main está alinhada com origin/main. O commit mais recente observado é "Finaliza reestruturacao dos modulos de aventura e personagem". As pastas de saída e arquivos temporários locais não pertencem ao produto.

27. Bloqueios reais para lançamento

Os bloqueios principais deixaram de ser visuais. Eles estão no fechamento do ciclo de jogo e na cobertura funcional do recorte prometido.

Prioridade	Bloqueio	Critério para considerar resolvido
1	Persistência pós-combate	PV, recursos, condições, recompensas e XP retornam ao personagem e permanecem após recarregar.
2	Transição combate-aventura	Vitória e derrota conduzem à cena correta, atualizam memórias e não duplicam consequências.
3	Guerreiro nível 1	Segundo Fôlego, usos, recuperação, equipamentos, armas, maestrias e talentos ofertados funcionam juntos.
4	A Fuga	A aventura pode ser concluída do início ao fim, incluindo caminhos, testes, batalhas e encerramento.
5	Classes restantes	Mago, Ladino e Clérigo nível 1 atendem aos critérios completos do recorte ofertado.
6	Qualidade pública	Responsividade, acessibilidade, revisão editorial, testes finais e licenças visuais estão documentados.

Sistemas ainda incompletos

- Motor genérico de efeitos: alcance, área, duração, condições e cobertura ampla de gatilhos.
- Economia de ações: reação, ação livre, recursos e sincronização completa entre interface e estado.
- Armas, maestrias, talentos e habilidades: cobertura completa apenas do que for oferecido ao jogador.
- Magias: consumo de espaços, concentração, ataque, salvaguarda, cura, duração e área.
- Progressão: conceder 100 XP uma única vez e preparar o estado para evolução futura.
- Espécies e criação: concluir todos os traços aplicáveis ao recorte exibido.
- Automação de testes: ainda não há uma bateria suficiente para substituir a validação manual integral.

Bloqueio de distribuição

As origens, autorias e licenças dos recursos visuais precisam ser registradas antes de uma distribuição pública ampla. Isso não impede testes privados ou uma Alpha fechada.

28. Estratégia de lançamento em duas metas

A recomendação atual é não esperar pelas quatro classes para validar o produto. Primeiro deve existir uma experiência vertical pequena, completa e persistente. Depois essa base será ampliada até o MVP formal definido pela checklist.

Meta A - Alpha vertical jogável

- Criação e salvamento de personagem.
- Somente Guerreiro nível 1; opções incompletas ficam escondidas ou claramente indisponíveis.
- A Fuga completa, com pelo menos um combate integrado.
- Vitória, derrota, retorno à narrativa e encerramento da aventura.
- PV, recursos, memórias, recompensas e 100 XP persistidos.
- Fechar e reabrir sem perder progresso nem duplicar recompensas.
- Uso integral sem console ou comandos manuais.

Meta B - MVP formal

- Mago, Ladino e Clérigo nível 1 completos.
- Conjuração básica, áreas, condições e concentração necessárias ao conteúdo ofertado.
- Motor de efeitos e economia de ações ampliados.
- A Fuga testada com todas as classes disponíveis.
- Rodada final de acessibilidade, responsividade, conteúdo, testes e licenças.

Decisão recomendada

A Alpha vertical deve ser tratada como instrumento de validação e teste, não como substituta silenciosa do MVP formal. O escopo formal continua contendo as quatro classes enquanto a checklist não for alterada.

Por que essa ordem

Fechar uma aventura com uma classe valida criação, ficha, narrativa, dados, combate, persistência e retorno em um único percurso. Construir primeiro todas as classes aumentaria o volume de regras antes de provar que o ciclo central funciona como produto.

29. Rota recomendada de desenvolvimento

Etapa	Objetivo	Saída verificável
1/6	Fechar o contrato aventura-combate	Definir e persistir PV, recursos, condições, resultado, recompensa, XP, próxima cena e memórias.
2/6	Concluir Guerreiro nível 1	Tudo o que a interface oferece ao Guerreiro funciona e persiste.
3/6	Terminar A Fuga	Todos os caminhos necessários, combates, vitória, derrota e encerramento podem ser jogados.
4/6	Salvar e retomar	Reabrir durante ou após a aventura preserva estado e não duplica consequências.
5/6	Estabilizar a Alpha	Bateria manual formal, correções de fluxo, textos, responsividade e opções incompletas ocultas.
6/6	Expandir ao MVP formal	Implementar e testar Mago, Ladino e Clérigo sobre a base validada.

Próxima intervenção - etapa 1/6

Antes de escrever muitas cenas novas ou retomar mudanças visuais, mapear o estado que entra no combate, o estado que muda durante o encontro e o estado que precisa retornar à aventura. A implementação deve ter uma única função de consolidação, idempotente, para impedir recompensas e XP duplicados.

Critérios de aceitação da etapa 1/6

- A vitória retorna à cena configurada e registra a memória correta.
- A derrota segue o destino configurado sem preservar um estado impossível.
- PV e recursos usados no combate aparecem corretamente na ficha após o retorno.
- 100 XP é concedido uma única vez quando aplicável.
- Recarregar a página não reaplica o resultado do combate.
- O mesmo contrato funciona para combates futuros sem lógica exclusiva de A Fuga.

Orientação para conteúdo

A escrita de A Fuga deve ser retomada na etapa 3/6. Pequenos trechos podem ser usados antes disso como casos de teste, mas a expansão narrativa principal deve esperar o contrato de persistência e retorno, para evitar retrabalho.

30. Registro consolidado e retomada

Campo	Registro
Data-base	18 de agosto de 2026
Frente concluída	Reestruturação modular de personagem, dados, câmera, combate, aventura e criação.
Testes confirmados	Criação, salvamento, entrada na aventura, iniciativa, ataque, dano, turnos, Segundo Fôlego, modificadores, resultado de dados, exclusão por botão direito, pan e zoom.
Checklist	146 de 394 itens concluídos (37,06%).
Git	main alinhada com origin/main; commit observado: 66cd3c1.
Decisão	Priorizar Alpha vertical com Guerreiro antes da expansão para quatro classes.
Próximo passo	Etapa 1/6: contrato e persistência do ciclo aventura-combate.

Resumo executivo para a próxima conversa

Onde estamos

A base técnica está madura, mas o produto ainda não fecha de forma persistente o ciclo criação - aventura - combate - consequência - retomada. A prioridade não é um novo redesenho visual nem ampliar imediatamente as classes; é concluir esse ciclo com o Guerreiro e terminar A Fuga como Alpha vertical.

- Ler este suplemento junto às versões anteriores, mas usar código e checklist atuais como fonte de verdade.
- Não reverter a modularização recente nem recriar responsabilidades nos antigos arquivos coordenadores.
- Manter dados descritivos, motores interpretativos, interfaces de apresentação e persistência separados.
- Tratar a próxima tarefa como 1/6 até que todos os critérios de aceitação do contrato aventura-combate estejam confirmados.
- Depois de cada etapa, atualizar checklist, testes confirmados, commit e este registro cumulativo.

FIM DO SUPLEMENTO 3.0